

TUGAS UAS WEEK 15

ROBOTIKA

Nama : Augryes Farha Sulistya Negara

Kelas : TK-45-01

NIM :1103210115

1. Pendahuluan

Robot Operating System (ROS) adalah framework fleksibel yang digunakan untuk pengembangan perangkat lunak robotika. Praktikum ini bertujuan memberikan pemahaman dasar tentang ROS, mempelajari struktur dan elemen utamanya, membuat model robot 3D, serta melakukan simulasi menggunakan Gazebo.

2. Tujuan

1. Memahami dasar-dasar ROS, termasuk struktur dan komponennya.
2. Membuat workspace, package, dan node sederhana di ROS.
3. Membuat model robot menggunakan URDF dan memvisualisasikannya di RViz.
4. Mensimulasikan robot menggunakan Gazebo dan mengontrolnya melalui ROS.

3. Alat dan Bahan

1. Sistem operasi: Ubuntu 20.04.
 2. ROS Noetic dengan komponen berikut:
 - o ros-noetic-desktop-full
 - o ros-noetic-gazebo-ros-pkgs
 - o ros-noetic-teleop-twist-keyboard
 3. Editor teks: VS Code, Nano, atau editor lainnya.
 4. Gazebo untuk simulasi lingkungan.
-

4. Prosedur

Chapter 1: Pengenalan ke ROS

1. Apa itu ROS? ROS adalah framework untuk pengembangan perangkat lunak robotika yang menyediakan alat dan pustaka untuk komunikasi antar modul robot. Elemen utama ROS meliputi:

- Node: Proses individual yang menjalankan fungsi tertentu.
- Topic: Saluran komunikasi antar node.
- Master: Pusat komunikasi ROS.

2. Langkah-langkah:

- Instalasi ROS Noetic:

bash

Copy code

sudo apt update

sudo apt install ros-noetic-desktop-full

- Menjalankan ROS Master:

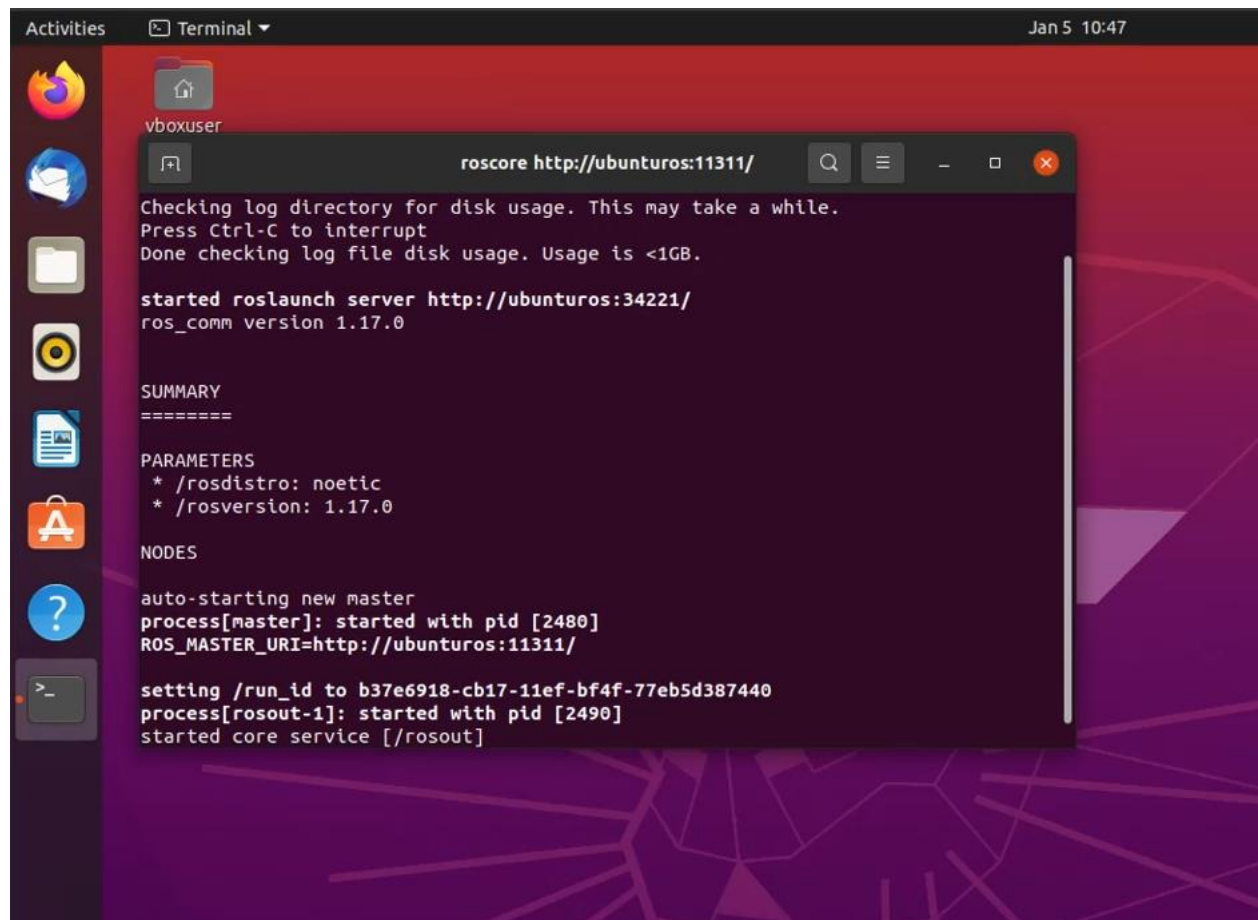
bash

Copy code

roscore

3. Hasil:

- ROS Master berhasil dijalankan dengan perintah roscore.



Chapter 2: Memulai Pemrograman dengan ROS

1. Membuat Workspace dan Package:

- Membuat workspace:

bash

Copy code

```
mkdir -p ~/catkin_ws/src
```

```
cd ~/catkin_ws
```

```
catkin_make
```

- Membuat package:

bash

Copy code

```
cd ~/catkin_ws/src
```

```
catkin_create_pkg my_first_package std_msgs rospy roscpp
```

2. Menulis Node ROS:

- Membuat file talker.py di package:

python

Copy code

```
#!/usr/bin/env python3
```

```
import rospy
```

```
from std_msgs.msg import String
```

```
def talker():
```

```
    pub = rospy.Publisher('chatter', String, queue_size=10)
```

```
    rospy.init_node('talker', anonymous=True)
```

```
    rate = rospy.Rate(10)
```

```
    while not rospy.is_shutdown():
```

```
        hello_str = f"Hello ROS {rospy.get_time()}"
```

```
        rospy.loginfo(hello_str)
```

```
        pub.publish(hello_str)
```

```
        rate.sleep()
```

```
if __name__ == '__main__':
```

```
    try:
```

```
        talker()
```

```
    except rospy.ROSInterruptException:
```

```
        pass
```

3. Menjalankan Node:

- Berikan izin eksekusi pada file:

bash

Copy code

`chmod +x src/talker.py`

- Jalankan node:

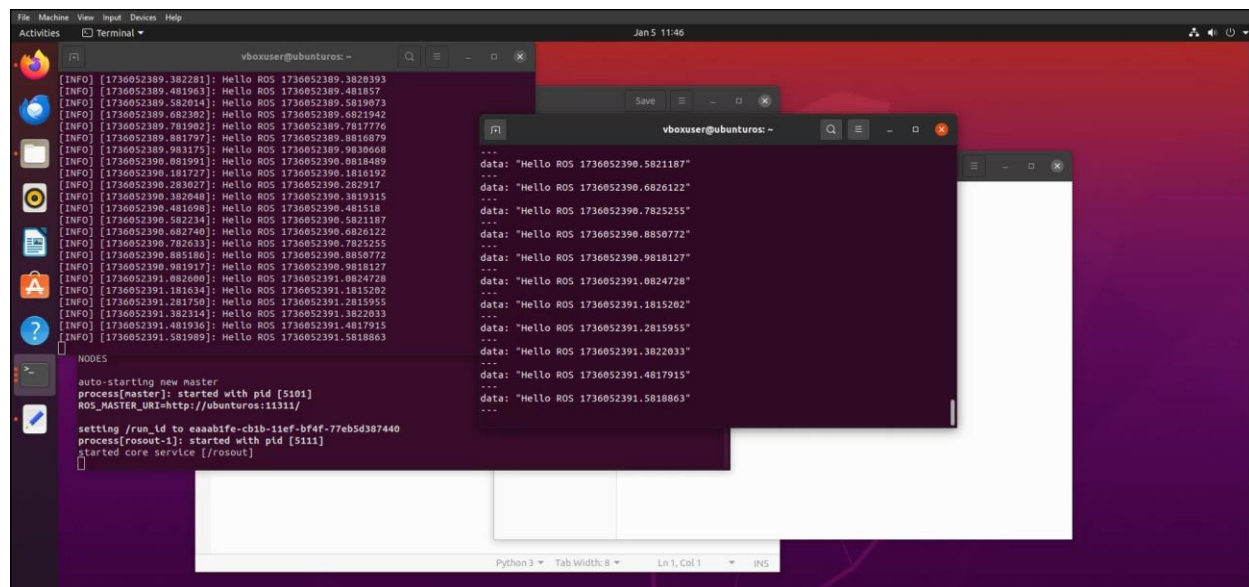
`bash`

Copy code

`roslaunch my_first_package talker.py`

4. Hasil:

- Node berhasil mengirimkan pesan ke topic `/chatter`, yang dapat dilihat dengan `rostopic echo /chatter`.



The screenshot shows a terminal window with two tabs. The left tab, titled 'vboxuser@ubuntu: ~', displays the output of a ROS node, showing a series of 'Hello ROS' messages with unique IDs. The right tab, titled 'vboxuser@ubuntu: ~', shows the output of the `rostopic echo /chatter` command, displaying the received messages. The terminal window is running on a Ubuntu system, as indicated by the desktop background and the terminal prompt.

Chapter 3: Membuat Model Robot dengan URDF

1. Langkah Membuat File URDF:

- Membuat package untuk deskripsi robot:

`bash`

Copy code

`cd ~/catkin_ws/src`

`catkin_create_pkg my_robot_description`

`mkdir -p ~/catkin_ws/src/my_robot_description/urdf`

- Menulis file `robot.urdf`:

xml

Copy code

```
<robot name="my_robot">
  <link name="base_link">
    <visual>
      <geometry>
        <box size="1 1 0.2"/>
      </geometry>
      <material name="blue">
        <color rgba="0 0 1 1"/>
      </material>
    </visual>
  </link>
</robot>
```

2. Memvalidasi dan Memvisualisasikan Robot:

- Validasi URDF:

bash

Copy code

```
check_urdf ~/catkin_ws/src/my_robot_description/urdf/robot.urdf
```

- Memuat URDF di RViz:

bash

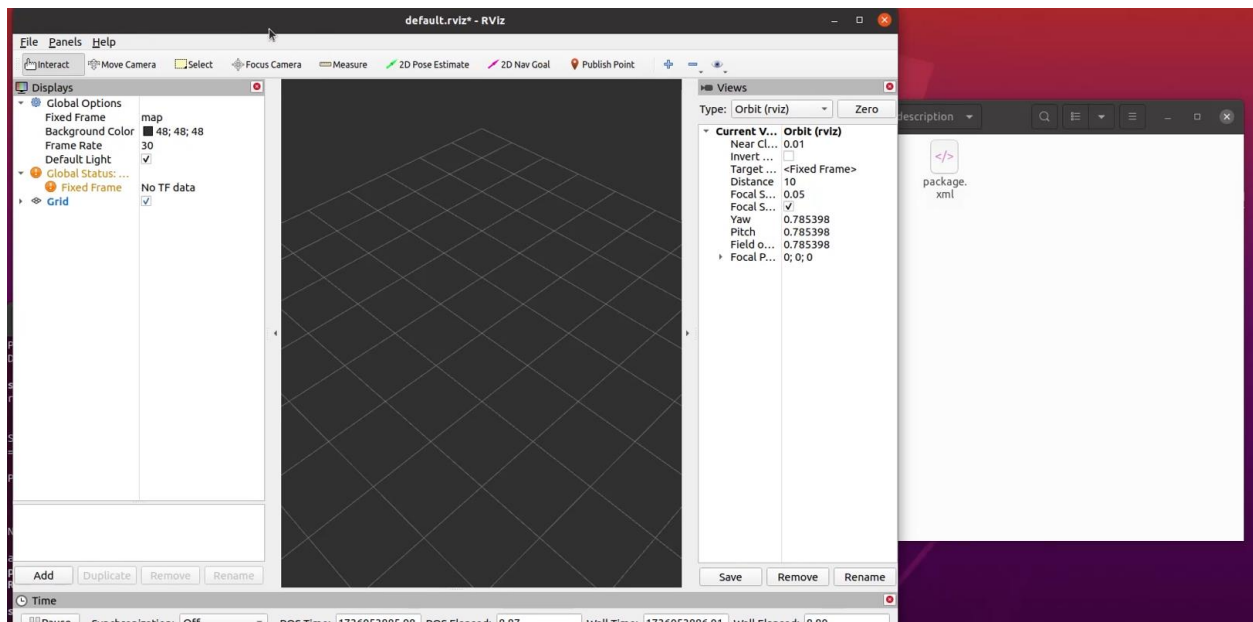
Copy code

```
rosparam set /robot_description -t ~/catkin_ws/src/my_robot_description/urdf/robot.urdf
```

```
roslaunch rviz rviz
```

3. Hasil:

- Robot berhasil divisualisasikan di RViz dengan model 3D sesuai deskripsi URDF.



Chapter 4: Simulasi Robot dengan Gazebo

1. Membuat Package Simulasi:

- Membuat package untuk simulasi:

bash

Copy code

```
cd ~/catkin_ws/src
```

```
catkin_create_pkg my_robot_simulation rospy roscpp gazebo_ros
```

```
mkdir -p ~/catkin_ws/src/my_robot_simulation/urdf
```

```
mkdir -p ~/catkin_ws/src/my_robot_simulation/worlds
```

2. Menambahkan File World Gazebo:

- Menulis file my_world.world:

xml

Copy code

```
<sdf version="1.6">
```

```
<world name="default">
```

```
<include>
```

```
<uri>model://ground_plane</uri>
```

```
</include>
```

```
<include>

  <uri>model://sun</uri>

</include>

</world>

</sdf>
```

3. Menjalankan Simulasi:

- o Jalankan Gazebo:

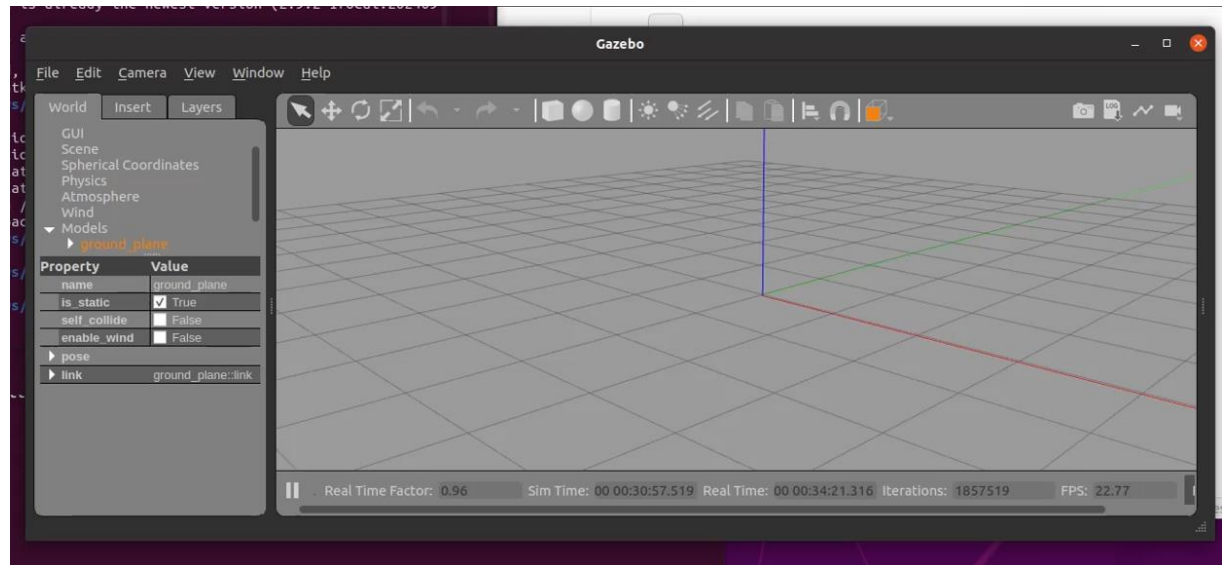
bash

Copy code

```
gazebo ~/catkin_ws/src/my_robot_simulation/worlds/my_world.world
```

4. Hasil:

- o Model robot berhasil dimuat di lingkungan Gazebo.



5. Hasil dan Pembahasan

1. Node sederhana berhasil dibuat dan dijalankan, menunjukkan komunikasi antar node di ROS.
 2. Model robot berhasil dibuat menggunakan URDF dan divisualisasikan di RViz.
 3. Simulasi robot di Gazebo berhasil dengan kontrol melalui ROS.
-

6. Kesimpulan

Melalui praktikum ini, pemahaman dasar tentang ROS, pemrograman node, pembuatan model 3D menggunakan URDF, serta simulasi robot di Gazebo berhasil dicapai. Framework ROS terbukti efektif dalam pengembangan dan pengujian aplikasi robotika.